



3. PROJETO DE SOFTWARE

3.9. MÉTODOS ÁGEIS: XP E SCRUM

DISCIPLINA: ENGENHARIA DE SOFTWARE I
CURSO: BACHAREL EM SISTEMAS DE INFORMAÇÃO
PROFESSOR: M.E JULLIAN HERMANN CREUTZBERG
2025

O que
sabemos
sobre
métodos
ágeis?



MÉTODOS ÁGEIS

- Não se trata apenas de ser uma forma “ágil” de trabalhar em relação a outras metodologias tradicionais “pesadas”
- Como sabemos para se desenvolver software precisamos de um conjunto de atividades e tarefas definidas que, quando colocadas em uso podemos chamar de metodologia (ou processo):
 - Cascata, Iterativa, Incremental, Espiral, RAD, etc.

3

MÉTODOS ÁGEIS

- Algumas metodologias exigem mais (ou menos) documentos, mais (ou menos) iteratividade (liberações ao cliente)
- Independentemente da metodologia utilizada algumas atividades são comuns:
 - Especificação;
 - Projeto e implementação;
 - Verificação, Validação & Teste;
 - Manutenção → Evolução;

4

MÉTODOS ÁGEIS

- Algumas organizações não tem uma metodologia definida, por inúmeras razões
- O resultado da falta de uma sistematização na produção de software é a baixa qualidade do que é feito, dificuldade na entrega do software (custo x prazo), dificuldade na evolução posterior, etc.

5

MÉTODOS ÁGEIS

- O termo metodologias ágeis tornou-se mais popular quando em 2001, 17 especialistas em processos de desenvolvimento estabeleceram alguns princípios, sendo este conjunto denominado: **manifesto ágil**

- <https://agilemanifesto.org/>
- <https://agilemanifesto.org/iso/ptbr/manifesto.html>



DESENVOLVIMENTO ÁGIL DE SOFTWARE



DESENVOLVIMENTO RÁPIDO DE SOFTWARE

- Atualmente, a entrega e o desenvolvimento rápidos têm sido geralmente, o requisito mais importante nos sistemas de software:
- Os negócios operam com requisitos que mudam rapidamente e é praticamente impossível produzir um conjunto estável de requisitos de software.
- O software precisa evoluir rapidamente para refletir as necessidades de negócio em constante mudança.

DESENVOLVIMENTO RÁPIDO DE SOFTWARE

■ Desenvolvimento rápido de software:

- A especificação, o projeto e a implementação são intercaladas.
- O sistema desenvolvido é entregue como uma série de versões, com os stakeholders envolvidos na avaliação das versões.
- Geralmente as interfaces de usuário são desenvolvidas usando uma IDE e um conjunto de ferramentas gráficas.

9

MÉTODOS ÁGEIS

- A insatisfação com as despesas que envolvem os métodos tradicionais de projeto de software dos anos de 1980 e 1990, levou a criação de métodos ágeis.
- Os métodos ágeis:
 - Têm foco no código ao invés de no projeto.
 - São baseados em uma abordagem iterativa de desenvolvimento de software.
 - São planejados para entregar rapidamente o software em funcionamento e evoluí-lo rapidamente para alcançar os requisitos em constante mudança.
- O objetivo dos métodos ágeis é reduzir os custos nos processos de software (ex. limitando a documentação) e permitir uma resposta rápida aos requisitos em constante mudança sem retrabalho excessivo.

10

MANIFESTO ÁGIL

- *“Estamos descobrindo melhores formas de desenvolver softwares e ajudar outros a fazê-lo também. Através desse trabalho, valorizamos mais:*
 - *Indivíduos e interações, ao invés de processos e ferramentas.*
 - *Softwares que já funcionam ao invés de documentação abrangente.*
 - *Colaboração do cliente ao invés de negociação contratual.*
 - *Resposta a mudanças ao invés de seguir um plano.*
- *O que significa que existe valor nos itens a direita, mas que valorizamos mais os itens a esquerda.”*

11

OS PRINCÍPIOS DOS MÉTODOS ÁGEIS

Princípios	Descrição
Envolvimento do cliente	Os clientes devem estar intimamente envolvidos no processo de desenvolvimento. Seu papel é fornecer e priorizar novos requisitos do sistema e avaliar suas iterações.
Entrega incremental	O software é desenvolvido em incrementos com o cliente, especificando os requisitos para serem incluídos em cada um.
Pessoas, não processos	As habilidades da equipe de desenvolvimento devem ser reconhecidas e exploradas. Membros da equipe devem desenvolver suas próprias maneiras de trabalhar, sem processos prescritivos.
Aceitar as mudanças	Deve-se ter em mente que os requisitos do sistema vão mudar. Por isso, projete o sistema de maneira a acomodar essas mudanças.
Manter a simplicidade	Focalize a simplicidade, tanto do software a ser desenvolvido quanto do processo de desenvolvimento. Sempre que possível, trabalhe ativamente para eliminar a complexidade do sistema.

12

APLICABILIDADE DOS MÉTODOS ÁGEIS

- Desenvolvimento de produto, quando a empresa de software está desenvolvendo um produto pequeno ou médio para venda.
- Desenvolvimento de sistema personalizado dentro de uma organização, quando existe um compromisso claro do cliente em se envolver no processo de desenvolvimento e quando não existem muitas regras e regulamentos externos que afetam o software.
- Devido ao foco em equipes pequenas e fortemente integradas, podem existir problemas na escalabilidade de métodos ágeis em sistemas grandes.

13



PROBLEMAS COM MÉTODOS ÁGEIS

- Pode ser difícil manter o interesse dos clientes que estão envolvidos no processo.
- Membros da equipe podem não ser adequados ao envolvimento intenso que caracteriza os métodos ágeis.
- Priorizar mudanças pode ser difícil onde existem múltiplos stakeholders.
- Manter a simplicidade requer trabalho extra.
- Os contratos podem ser um problema assim como em outras abordagens que usam o desenvolvimento iterativo.

14

MÉTODOS ÁGEIS E MANUTENÇÃO DE SOFTWARE

- A maioria das organizações gasta mais na manutenção de softwares existentes do que no desenvolvimento de softwares novos.
- Devido a isso, para que os métodos ágeis obtenham sucesso, os softwares devem receber tanta manutenção quanto o desenvolvimento original.

15

MÉTODOS ÁGEIS E MANUTENÇÃO DE SOFTWARE

- **Duas questões muito importantes:**
 - É possível dar suporte aos sistemas que são desenvolvidos usando uma abordagem ágil, tendo em vista a ênfase no processo de minimização da documentação formal?
 - Os métodos ágeis podem ser usados efetivamente, para evoluir um sistema em resposta a mudanças nos requisitos do cliente?
- Podem ocorrer problemas no caso do tempo original de desenvolvimento não puder ser mantido.

16

DESENVOLVIMENTO ÁGIL X DIRIGIDO A PLANOS

■ Desenvolvimento dirigido a planos

- Para a engenharia de software, uma abordagem dirigida a planos, é baseada em estágios de desenvolvimento separados, com os produtos a serem produzidos em cada um desses estágios planejados antecipadamente.
- O desenvolvimento incremental é possível também no dirigido a planos.

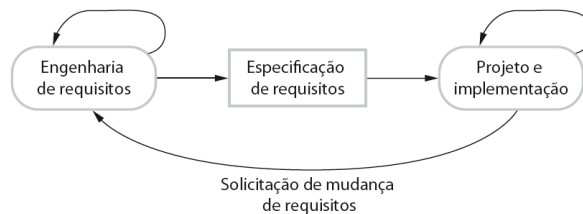
■ Desenvolvimento ágil

- Especificação, projeto, implementação e teste são intercalados e os produtos do processo de desenvolvimento são decididos através de um processo de negociação, durante o processo de desenvolvimento do software.

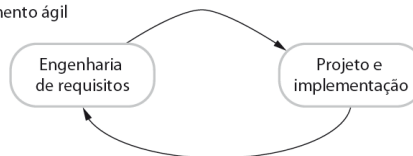
17

ESPECIFICAÇÕES DIRIGIDA A PLANOS E ÁGIL

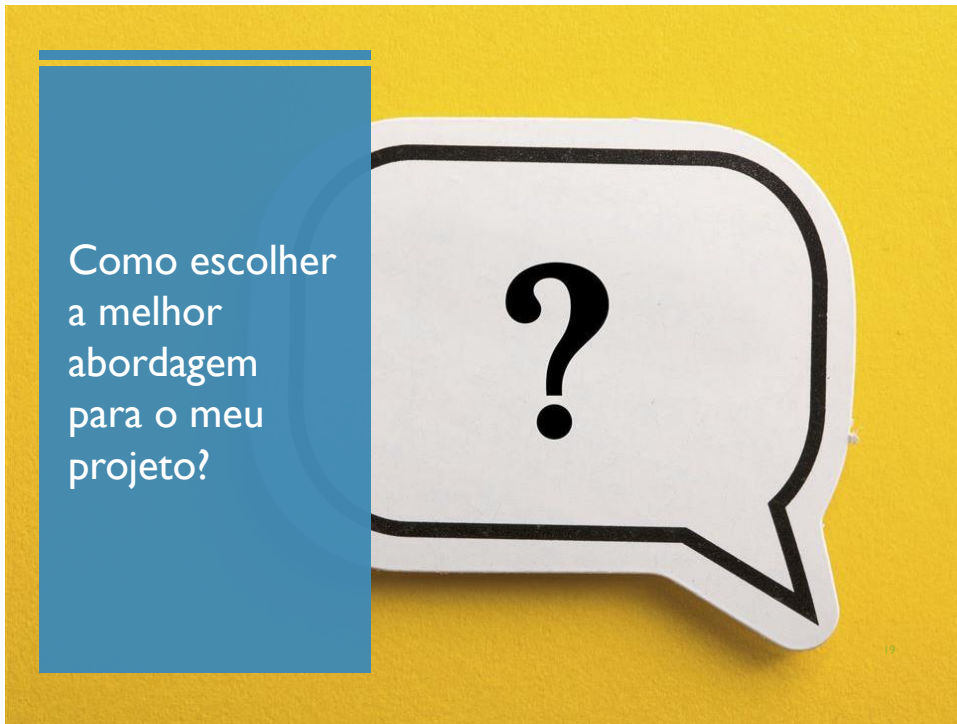
Desenvolvimento baseado em planos



Desenvolvimento ágil



18



QUESTÕES TÉCNICAS, HUMANAS E ORGANIZACIONAIS

■ **A maioria dos projetos incluem elementos de processos dirigidos a planos e ágeis.**

■ **Decidir no **equilíbrio** depende de pelo menos 10 fatores:**

1. É importante ter uma especificação e projeto bem detalhados antes de passar para a implementação? Caso seja, provavelmente você precisa usar uma abordagem dirigida a planos.
2. Uma estratégia de entrega incremental onde você entrega o software para os clientes e recebe feedback rápido deles é possível? Caso seja, considere usar métodos ágeis.

20

QUESTÕES TÉCNICAS, HUMANAS E ORGANIZACIONAIS

3. Qual o tamanho do sistema a ser desenvolvido? Os métodos ágeis são mais efetivos quando o sistema pode ser desenvolvido com uma equipe pequena que pode se comunicar informalmente. O que pode não ser possível para sistemas grandes que requerem grandes equipes de desenvolvimento, nesses casos, deve ser usada uma abordagem dirigida a planos.
4. Que tipo de sistema está sendo desenvolvido? Abordagens dirigidas a planos podem ser necessárias para sistemas que requerem muita análise antes da implementação (ex. sistema que opere em tempo real com requisitos de temporização complexos).

21

QUESTÕES TÉCNICAS, HUMANAS E ORGANIZACIONAIS

5. Qual é o tempo de vida esperado para o sistema? Sistemas com longo tempo de vida podem precisar de mais documentação de projeto para comunicar as intenções originais dos desenvolvedores do sistema para a equipe de suporte.
6. Quais tecnologias estão disponíveis para manter o desenvolvimento do sistema? Métodos ágeis dependem de boas ferramentas para acompanhar um sistema em evolução.

22

QUESTÕES TÉCNICAS, HUMANAS E ORGANIZACIONAIS

7. Como está organizada a equipe de desenvolvimento? Se a equipe de desenvolvimento está distribuída ou se parte do desenvolvimento está sendo terceirizado você pode precisar desenvolver documentos de projeto para que haja comunicação entre as equipes de desenvolvimento.
8. Existem questões culturais ou organizacionais que podem afetar o desenvolvimento do sistema? As organizações tradicionais de engenharia têm uma cultura de desenvolvimento dirigido a planos, o que é padrão em engenharia.

23

QUESTÕES TÉCNICAS, HUMANAS E ORGANIZACIONAIS

9. O quão bons são os projetistas e os programadores da equipe de desenvolvimento? É dito que os métodos ágeis requerem um nível de habilidade mais alto do que as abordagens dirigidas a planos, nas quais os programadores simplesmente traduzem um projeto detalhado em código.
10. O sistema está sujeito a regulamentação externa? Se o sistema precisa ser aprovado por um regulador externo (ex. O FAA [Administração Federal de Aviação] aprova softwares críticos para a operação de um avião) então provavelmente requisitaram a você a produção de documentação detalhada como parte da documentação de segurança do sistema.

24



EXTREME PROGRAMMING



25



EXTREME PROGRAMMING

- O **Extreme Programming** (XP) usa uma abordagem 'extrema' ao desenvolvimento iterativo.
- Novas versões podem ser construídas várias vezes por dia;
- Incrementos são entregues aos clientes a cada 2 semanas;
- Todos os testes devem ser realizados em todas as versões e cada versão só é aceita se os testes forem concluídos com sucesso.

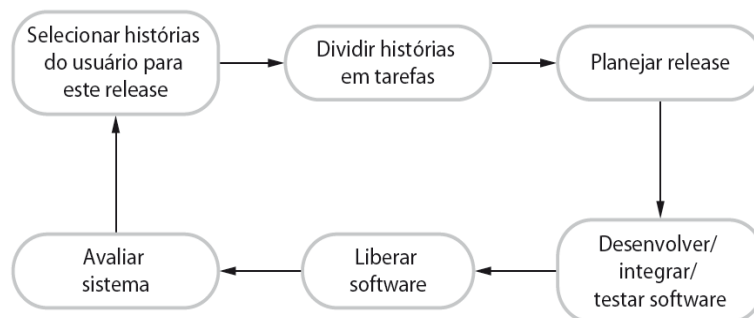
26

PRINCÍPIOS OS MÉTODOS ÁGEIS E DO XP

- O desenvolvimento incremental é mantido através de releases de sistema pequenos e frequentes.
- O envolvimento do cliente significa compromisso do cliente com a equipe em tempo integral.
- 'Pessoas e não processos' por meio de programação em pares, propriedade coletiva do código e um processo que evita longas horas de trabalho.
- Mudanças suportadas através de releases regulares de sistema.
- Manter a simplicidade através de constante refatoração de código.

27

O CICLO DE UM RELEASE EM EXTREME PROGRAMMING



28

PRÁTICAS DO EXTREME PROGRAMMING

Princípio ou prática	Descrição
Planejamento incremental	Os requisitos são gravados em cartões de história e as histórias que serão incluídas em um release são determinadas pelo tempo disponível e sua relativa prioridade. Os desenvolvedores dividem essas histórias em 'Tarefas'. ¹
Pequenos releases	Em primeiro lugar, desenvolve-se um conjunto mínimo de funcionalidades útil, que fornece o valor do negócio. Releases do sistema são frequentes e gradualmente adicionam funcionalidade ao primeiro release.
Projeto simples	Cada projeto é realizado para atender às necessidades atuais, e nada mais.
Desenvolvimento test-first	Um framework de testes iniciais automatizados é usado para escrever os testes para uma nova funcionalidade antes que a funcionalidade em si seja implementada.
Refatoração	Todos os desenvolvedores devem refatorar o código continuamente assim que encontrarem melhorias de código. Isso mantém o código simples e manutenível.

29

PRÁTICAS DO EXTREME PROGRAMMING

Princípio ou prática	Descrição
Programação em pares	Os desenvolvedores trabalham em pares, verificando o trabalho dos outros e prestando apoio para um bom trabalho sempre.
Propriedade coletiva	Os pares de desenvolvedores trabalham em todas as áreas do sistema, de modo que não se desenvolvam ilhas de <i>expertise</i> . Todos os conhecimentos e todos os desenvolvedores assumem responsabilidade por todo o código. Qualquer um pode mudar qualquer coisa.
Integração contínua	Assim que o trabalho em uma tarefa é concluído, ele é integrado ao sistema como um todo. Após essa integração, todos os testes de unidade do sistema devem passar.
Ritmo sustentável	Grandes quantidades de horas-extra não são consideradas aceitáveis, pois o resultado final, muitas vezes, é a redução da qualidade do código e da produtividade a médio prazo.
Cliente no local	Um representante do usuário final do sistema (o cliente) deve estar disponível todo o tempo à equipe de XP. Em um processo de Extreme Programming, o cliente é um membro da equipe de desenvolvimento e é responsável por levar a ela os requisitos de sistema para implementação.

30

CENÁRIOS DE REQUISITOS

- Em XP, um cliente ou usuário é parte do time de XP e é responsável na tomada de decisões sobre requisitos.
- Requisitos do usuário são expressos como cenários ou histórias dos usuários.
- Esses são escritos em cartões e a equipe de desenvolvimento os divide em tarefas de implementação.
- Essas tarefas são a base das estimativas de cronograma e custo.
- O cliente escolhe as histórias que serão incluídas no próximo release baseando-se nas suas prioridades e nas estimativas de cronograma.

31

EXEMPLO UMA ESTÓRIA DE 'PRESCRIÇÃO DE MEDICAMENTOS'

Prescrição de medicamentos

Kate é uma médica que deseja prescrever medicamentos para um paciente de uma clínica. O prontuário do paciente já está sendo exibido em seu computador, assim, ela clica o campo 'medicação' e pode selecionar 'medicação atual', 'nova medicação', ou 'formulário'.

Se ela selecionar 'medicação atual', o sistema pede que ela verifique a dose. Se ela quiser mudar a dose, ela altera esta e em seguida, confirma a prescrição.

Se ela escolher 'nova medicação', o sistema assume que ela sabe qual medicação receitar.

Ela digita as primeiras letras do nome do medicamento. O sistema exibe uma lista de possíveis fármacos que começam com essas letras. Ela escolhe a medicação requerida e o sistema responde, pedindo-lhe para verificar se o medicamento selecionado está correto.

Ela insere a dose e, em seguida, confirma a prescrição.

Se ela escolhe 'formulário', o sistema exibe uma caixa de busca para o formulário aprovado.

Ela pode, então, procurar pelo medicamento requerido. Ela seleciona um medicamento e é solicitado que verifique se a medicação está correta. Ela insere a dose e, em seguida, confirma a prescrição.

O sistema sempre verifica se a dose está dentro da faixa permitida. Caso não esteja, Kate é convidada a alterar a dose.

Após Kate confirmar a prescrição, esta será exibida para verificação. Ela pode escolher 'OK' ou 'Alterar'. Se clicar em 'OK', a prescrição fica gravada nos bancos de dados da auditoria.

Se ela clicar em 'Alterar', reinicia o processo de 'Prescrição de Medicamentos'.

EXEMPLO**CARTÕES DE TAREFA PARA A 'PRESCRIÇÃO DE MEDICAMENTOS'**

Tarefa 1: Alterar dose de medicamentos prescritos

Tarefa 2: Seleção de formulário

Tarefa 3: Verificação de dose

A verificação da dose é uma precaução de segurança para verificar se o médico não receitou uma dose perigosamente pequena ou grande.

Usando o ID do formulário para o nome do medicamento genérico, procure o formulário e obtenha a dose mínima e máxima recomendada.

Verifique a dose mínima e máxima prescrita. Caso esteja fora da faixa, emita uma mensagem de erro dizendo que a dose está muito alta ou muito baixa.

Caso esteja dentro da faixa, habilite o botão 'Confirmar'.

33

XP E MUDANÇAS

- O senso comum da engenharia de software diz que se deve projetar pensando em mudanças.
- Vale a pena gastar tempo e esforço antecipando as mudanças já que, posteriormente, esse esforço reduz custos no ciclo de vida.
- No entanto, o XP afirma que isso não vale a pena já que as mudanças não podem ser antecipadas de forma confiável.
- Ao invés disso, propõe melhorias constantes do código (**refatoração**) para tornar as mudanças mais fáceis quando essas precisam ser implementadas.

34

REFATORAÇÃO

- A equipe de programação busca possíveis melhorias de software e as faz mesmo quando essas não são uma necessidade imediata.
- O que melhora a compreensão do software e reduz a necessidade de documentação.
- Torna-se mais fácil fazer mudanças porque o código é bem construído e limpo.
- No entanto, algumas mudanças requerem refatoração da arquitetura, o que é muito mais caro.

35

EXEMPLOS DE REFATORAÇÃO

- Reorganização de uma hierarquia de classes para remover código duplicado.
- Organização e renomeação de atributos e métodos para torná-los mais fáceis de entender.

36

TESTES EM XP

- Em XP, os testes são fundamentais, XP desenvolveu uma abordagem em que o programa é testado depois de que cada alteração é feita.
- **Características de testes em XP:**
 - Desenvolvimento test-first.
 - Desenvolvimento de testes incrementais a partir de cenários.
 - Envolvimento do usuário no desenvolvimento de testes e validação.
 - Cada vez que um novo release é construído, são usados frameworks de testes automatizados para executarem todos os testes de componentes.



DESENVOLVIMENTO TEST-FIRST

- Escrever testes antes do código esclarece os requisitos que devem ser implementados.
- Os testes são escritos na forma de programas ao invés de dados para que possam ser executados automaticamente.
- Os testes incluem checagem de que foram executados corretamente.
- Geralmente conta com um framework de testes.
- Todos os testes anteriores e novos são executados automaticamente quando uma nova funcionalidade é adicionada, para checar se a nova funcionalidade não introduziu erros.

38

ENVOLVIMENTO DO CLIENTE NOS TESTES

- A função do cliente no processo de testes é ajudar a desenvolver testes de aceitação para as histórias que serão implementadas no próximo release do sistema.
- O cliente, parte da equipe, escreve testes conforme o desenvolvimento prossegue.
- Todo código novo é validado para garantia de que seja o que o cliente precisa.
- No entanto, a pessoa que assume a função de cliente tem tempo limitado disponível e não pode trabalhar em tempo integral com a equipe de desenvolvimento.
- Eles podem pensar que prover os requisitos seja contribuição suficiente³⁹ e se tornarem relutantes em se envolverem no processo de testes.

DESCRIÇÃO DE CASO DE TESTE PARA VERIFICAÇÃO DE DOSE

Teste 4: Verificação de dose

Entrada:

1. Um número em mg representando uma única dose da medicação.
2. Um número que representa o número de doses únicas por dia.

Testes:

1. Teste para entradas em que a dose única é correta, mas a frequência é muito alta.
2. Teste para entradas em que a única dose é muito alta e muito baixa.
3. Teste para entradas em que a dose única x frequência é muito alta e muito baixa.
4. Teste para entradas em que a dose única x frequência é permitida.

Saída:

Mensagem de OK ou erro indicando que a dose está fora da faixa de segurança.

A AUTOMAÇÃO DE TESTES

- A **automação de testes** significa que os testes são escritos como componentes executáveis antes que a tarefa seja implementada.
- Esses componentes de teste devem ser autômatos, devem simular a submissão de entrada para ser testada e devem avaliar se o resultado atende à especificação de saída.
- Um framework de testes automatizados (ex. Junit, PHP Unit) é um sistema que facilita a escrita de testes executáveis e a submissão de um conjunto de testes para execução.

41

A AUTOMAÇÃO DE TESTES

- Como os testes são automatizados, sempre existe um conjunto de testes que podem ser rapidamente e facilmente executados.
- Quando qualquer funcionalidade é adicionada ao sistema os testes podem ser executados e problemas que o novo código possa ter introduzido podem ser percebidos imediatamente.



42

DIFICULDADES DOS TESTES EM XP

- Os programadores preferem programar do que testar e as vezes eles usam atalhos quando escrevem esses testes.
 - Por exemplo, eles podem escrever testes incompletos que não avaliam todas as possíveis exceções que podem ocorrer.
- Alguns testes podem ser muito difíceis de serem escritos de forma incremental.
- É difícil julgar se um conjunto de testes está completo.
- Embora você tenha vários testes de sistema, o conjunto dos testes pode não prover uma cobertura completa.

43



PROGRAMAÇÃO EM PARES

- Em XP, programadores trabalham em pares sentando junto para desenvolver código.
- Isso ajuda a desenvolver propriedade coletiva do código e espalha o conhecimento na equipe.
- Serve como um processo de revisão informal pois cada linha do código é observada por mais de uma pessoa.

44

PROGRAMAÇÃO EM PARES

- Encoraja a refatoração pois toda a equipe pode se beneficiar dessa atividade.
- Avaliações sugerem que a produtividade do desenvolvimento com programação em pares é similar a de duas pessoas trabalhando independentemente.
- Na programação em pares os programadores sentam-se juntos na mesma estação de trabalho para desenvolver softwares.

45

PROGRAMAÇÃO EM PARES

- Os pares são criados dinamicamente para que todos os membros da equipe trabalhem com cada um dos outros membros durante o processo de desenvolvimento.
- O compartilhamento de conhecimento que acontece durante a programação em pares é muito importante por reduzir os riscos gerais de um projeto quando um membro da equipe vai embora.
- A programação em pares não é necessariamente ineficiente e existem evidências de que o trabalho em pares é mais eficiente do que 2 programadores trabalhando separadamente.

46

VANTAGENS DA PROGRAMAÇÃO EM PARES

- Apoiar a ideia da propriedade coletiva e responsabilidade pelo sistema.
- Os indivíduos não são responsabilizados por problemas no código.
- Ao invés disso, a equipe tem responsabilidade coletiva na solução desses problemas.

47

VANTAGENS DA PROGRAMAÇÃO EM PARES

- Funciona como um processo de revisão informal porque cada linha de código é observada por pelo menos duas pessoas.
- Ajuda a apoiar a refatoração, que é um processo de melhoria do software.
- Em processos nos quais a programação em pares e a propriedade coletiva são usados, outros se beneficiam imediatamente da refatoração, o que provavelmente fará com que apoiem o processo.

48

PONTOS IMPORTANTES

- Os métodos ágeis são métodos de desenvolvimento incremental centrados no desenvolvimento rápido, frequentes releases de software, redução de custos de processo e produção de código de alta qualidade.
- Eles envolvem o cliente diretamente no processo de desenvolvimento.

49

PONTOS IMPORTANTES

- A decisão de quando usar uma abordagem ao desenvolvimento ágil ou dirigida a planos deve depender do tipo de software que está sendo desenvolvido, da capacidades da equipe de desenvolvimento e da cultura da empresa desenvolvedora do sistema.
- O Extreme Programming é um método ágil que integra uma série de boas práticas de programação, como por exemplo, releases de software frequentes, melhorias contínuas de software e participação do cliente na equipe de desenvolvimento.

50

SCRUM

51

GERENCIAMENTO ÁGIL DE PROJETOS

- A principal responsabilidade de gerentes de projeto de software é gerenciar o projeto para que o software seja entregue em tempo e dentro do orçamento planejado para o projeto.
- A abordagem padrão para o gerenciamento de projeto é dirigida a planos.
- Os gerentes estruturam um plano para o projeto mostrando o que deve ser entregue, quando deve ser entregue e quem irá trabalhar no desenvolvimento dos entregáveis.
- O gerenciamento ágil de projetos requer uma abordagem diferente, adaptada ao desenvolvimento incremental e aos pontos fortes particulares dos métodos ágeis.
- **Uma das abordagens mais conhecidas é o SCRUM.**

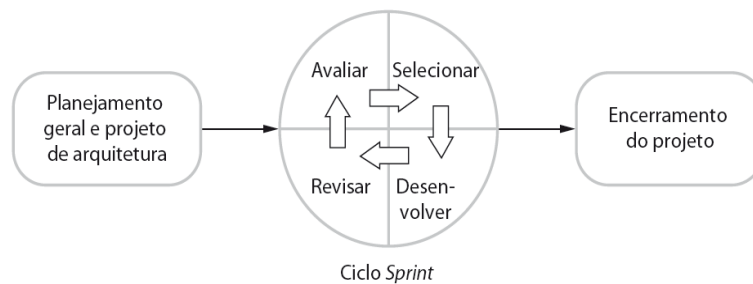
52

SCRUM

- A abordagem Scrum é um método ágil genérico mas seu foco é na gerência de desenvolvimento iterativo ao invés de práticas ágeis específicas.
- Existem três fases no Scrum:
 1. A **fase inicial** é uma fase de planejamento em que se estabelece os objetivos gerais do projeto e se projeta a arquitetura do software.
 2. Essa é seguida por uma série de **ciclos de Sprint**, em que cada ciclo desenvolve um incremento do sistema.
 3. A **fase de encerramento** do projeto finaliza o projeto, completa a documentação necessária como frames de ajuda do sistema e manuais de usuário e avalia as lições aprendidas no projeto.

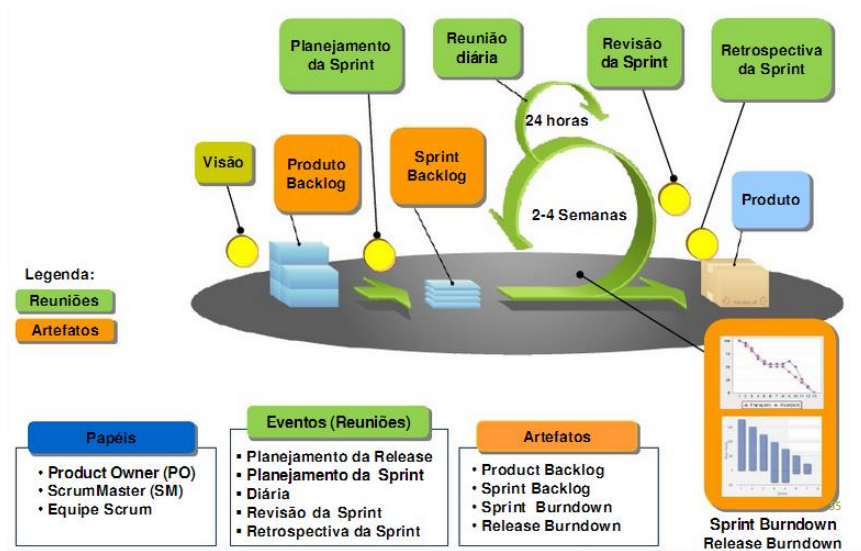
53

O PROCESSO SCRUM



54

SCRUM



O CICLO DE SPRINT

- Os **Sprints** possuem um deadline definido, geralmente de 2 a 4 semanas.
- Eles correspondem ao desenvolvimento de um release de um sistema em XP.
- O ponto de partida de planejamento é o **backlog de produto**, que é a lista de trabalho a ser feito no projeto.
- A fase de seleção envolve a seleção das características e funções que serão desenvolvidas durante o Sprint, pela equipe do projeto que trabalha com o cliente, chamado de **sprint backlog**.

56

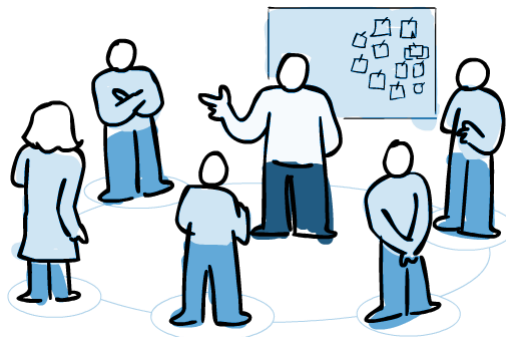
O CICLO DE SPRINT

- Assim que isso é definido, a equipe se organiza para desenvolver o software.
- Durante esse estágio a equipe é isolada do cliente e da organização, com todas as comunicações canalizadas por meio do chamado **Scrum Master**.
- A função do Scrum Master é proteger a equipe de desenvolvimento de distrações externas.
- Ao final do Sprint o trabalho feito é revisto e apresentado aos stakeholders.
- Assim o próximo ciclo de Sprint começa.

57

TRABALHO EM EQUIPE NO SCRUM

- O Scrum Master é um facilitador que organiza reuniões diárias, mantém o backlog do trabalho a ser feito, grava decisões, mede o processo usando o backlog e comunica-se com os clientes e a gerência fora da equipe.



58

TRABALHO EM EQUIPE NO SCRUM

- A equipe inteira comparece às **reuniões diárias** curtas nas quais todos os membros da equipe compartilham informações, descrevem seu progresso desde a última reunião, descrevem os problemas que surgiram e o que está planejado para o dia seguinte.
- Com isso, todos na equipe sabem o que está acontecendo e, caso ocorra um problema, podem replanejar o trabalho a curto prazo para lidar com a situação.



59

BENEFÍCIOS DO SCRUM

- O produto é dividido em um conjunto de partes gerenciáveis e compreensíveis.
- Requisitos instáveis não impedem o progresso.
- Toda a equipe tem visão de tudo e consequentemente a comunicação da equipe é melhorada.
- Os clientes recebem a entrega dos incrementos no tempo certo, além do feedback de como o produto funciona.
- Se estabelece a confiança entre os clientes e os desenvolvedores e se cria uma cultura positiva na qual todos acham que o projeto dará certo.

60



ESCALAMENTO DE MÉTODOS ÁGEIS

- Os métodos ágeis provaram-se bem-sucedidos para projetos pequenos e médios que podem ser desenvolvidos por uma equipe pequena e localizada.
- É dito que o sucesso desses métodos ocorre devido a melhorias na comunicação, as quais são possíveis quando todos estão trabalhando juntos.
- A escalamento dos métodos ágeis envolve mudá-los para que lidem com projetos maiores e mais longos onde existem múltiplas equipes de desenvolvimento, talvez trabalhando em localizações diferentes.

62

DESENVOLVIMENTO DE SISTEMAS DE GRANDE PORTE

- Geralmente, os sistemas de grande porte são coleções de sistemas separados que se comunicam, e nos quais as equipes desenvolvem cada sistema separadamente.
- Frequentemente essas equipes trabalham em locais diferentes, as vezes em fusos horários diferentes.
- Vários dos requisitos de sistema se preocupam com essa interação o que não permite flexibilidade e desenvolvimento incremental.
- Vários sistemas são integrados para criar um sistema, e uma fração significativa do desenvolvimento é voltada para a configuração do sistema ao invés do desenvolvimento do código original.

63

DESENVOLVIMENTO DE SISTEMAS DE GRANDE PORTE

- Os sistemas de grande porte e seus processos de desenvolvimento geralmente são restringidos por regras externas e regulamentações que limitam a forma como podem ser desenvolvidos.
- Os sistemas de grande porte tem um tempo de aquisição e desenvolvimento longo.
- Durante esse período, é difícil manter equipes coesas, que conhecem o sistema já que inevitavelmente as pessoas podem sair para outros trabalhos e projetos.
- Geralmente, os sistemas de grande porte tem um conjunto diversificado de stakeholders.
- É praticamente impossível envolver todos eles no processo de desenvolvimento.

64

ESCALAMENTO PARA SISTEMAS DE GRANDE PORTE

- A escalar métodos ágeis é essencial manter os **fundamentos ágeis**:
 - Planejamento flexível, releases de sistema frequentes, integração contínua, desenvolvimento dirigido a testes e boa comunicação entre os membros da equipe.

65

ESCALAMENTO PARA SISTEMAS DE GRANDE PORTE

- Para o desenvolvimento de sistemas de grande porte não é possível focar apenas no código do sistema.
- De início, é necessário fazer mais designs e documentação do sistema.
- Os mecanismos de comunicação entre as equipes precisam ser desenvolvidos e usados.
- O que deve envolver telefones comuns e videoconferências e reuniões virtuais curtas e frequentes entre os membros da equipe, nas quais as equipes se informam mutuamente acerca do progresso do trabalho.
- A integração contínua, na qual o sistema todo é construído cada vez que qualquer desenvolvedor aplica uma mudança, é praticamente impossível.
- No entanto, é essencial manter builds frequentes e releases regulares do sistema.

66

ESCALAMENTO PARA SISTEMAS DE GRANDE PORTE

- Gerentes de projeto que não possuem experiência em métodos ágeis podem ser relutantes em aceitar o risco de uma nova abordagem.
- Geralmente as grandes organizações possuem procedimentos e padrões de qualidade que espera-se que sejam seguidos por todos os projetos e, devido a sua natureza burocrática, são incompatíveis com os métodos ágeis.
- Os métodos ágeis parecem funcionar melhor quando os membros da equipe possuem um nível de competência relativamente alto.
- No entanto, dentro de grandes organizações, geralmente ocorre uma grande variação de competências e habilidades.
- Pode haver resistência cultural aos métodos ágeis, especialmente nessas organizações com um longo histórico de uso de processos convencionais da engenharia de sistemas.

67

PONTOS IMPORTANTES

- Um ponto particularmente forte da programação extrema é o desenvolvimento de testes automatizados antes de se criar um atributo do programa.
- Todos os testes devem ser executados com sucesso quando um incremento é integrado ao sistema.
- O método Scrum é um método ágil que provê um framework de gerenciamento de projeto. É baseado em um conjunto de Sprints, que são períodos fixos de tempo em que um incremento de sistema é desenvolvido.
- Escalamento de métodos ágeis para sistemas de grande porte é difícil. Tais sistemas precisam de mais projeto inicial e alguma documentação.

68

REFERÊNCIAS

- SOMMERVILLE, I. Engenharia de software. 9. ed. São Paulo: Pearson, 2011.
- Capítulo 3



✉ jullian@unidavi.edu.br

📷 [@jullianhc](https://www.instagram.com/jullianhc)